Rockchip GPIO 开发指南

文件标识：RK-YH-YF-803

发布版本：V1.0.1

日期：2025-11-28

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档主要介绍GPIO在Kernel和U-Boot中的使用方法，GPIO的Debug工具以及GPIO常见问题排查方法。

产品版本

芯片名称
Rockchip所有平台

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	沈 涛	2024-08-08	初始版本
V1.0.1	徐明浩	2025-11-28	添加了IO复用、读取电平异常、提高中断效率的方法

目录

Rockchip GPIO 开发指南

1. IO-Domain
   - 1.1 Kernel配置
   - 1.2 U-Boot配置
2. Kernel中开发GPIO
   - 2.1 GPIO复用
   - 2.2 GPIO输入输出
   - 2.3 动态设置IO复用
3. U-Boot中开发GPIO
   - 3.1 U-Boot LED驱动开发
   - 3.2 U-Boot GPIO驱动开发
4. 用户态开发GPIO
5. IO休眠唤醒设置
   - 5.1 配置GPIO唤醒
   - 5.2 配置休眠时GPIO电平
6. GPIO扩展芯片
7. GPIO模拟I2C
8. GPIO Debug工具
   - 8.1 IO工具
     - 8.1.1 Kernel阶段IO工具
     - 8.1.2 U-Boot阶段IO工具
   - 8.2 IOMUX工具
   - 8.3 Pinconfig工具
   - 8.4 PinDebug工具
9. GPIO 排查步骤
   - 9.1 电源确认
   - 9.2 IO复用确认
   - 9.3 GPIO CLK状态确认
   - 9.4 其他驱动调用GPIO导致冲突确认
   - 9.5 GPIO 寄存器确认
10. FAQ
    - 10.1 系统上电输出电平无法控制
    - 10.2 GPIO中断误触发
    - 10.3 RV1103/RV1106的gpio3b0~gpio3c3无法控制
    - 10.4 RV1108的gpio1a0~gpio1b1无法控制
    - 10.5 RK3588的gpio4d2、gpio4d3无法控制
    - 10.6 Android SDK没有找到/sys/class/gpio的系统层接口
    - 10.7 GPIO读取电平异常
    - 10.8 GPIO中断绑定指定核心
    - 10.9 GPIO中断响应延迟

1. IO-Domain

一般 IO 的输出电平有 1.8v，3.0v，3.3v等，不同的输出电压取决于IO所在的电源域电压，这个与硬件供电有关系。io-domain 是 IO 电源域寄存器的一个软件配置项，如果电源域软件配置跟硬件实际不匹配，会导致该电源域的IO口不受控制，出来的信号紊乱、不达标。如果在实际开发过程中，遇到某一路IO异常，查看复用状态没错也没别处调用，但是测量出来的信号就是不对，软件无法控制输出高低等，此时就需要排查该路的电源域寄存器配置。关于 io-domain 的详细开发指南，请查看SDK中自带的文档。

android sdk 文件路径：RKDocs\common\IO-Domain

linux sdk 文档路径：docs\cn\Common\IO-DOMAIN

注：有些SDK未包含 io-domain 的开发文档，请参考如下说明，或者找对应的FAE窗口获取。

特别需要注意的是RK新的芯片型号(如：RK3588 RK3562 RV1106 RV1103等)不需要软件配置io_domains，芯片内部可以自适配电压域，可以通过默认编译的EVB板固件，在kernel中编译保存的隐藏文件.xxx.dtb.dts.tmp确认，如果在tmp文件中没有搜索到io-domain配置项，说明软件就不需要去配置电压域。

1.1 Kernel配置

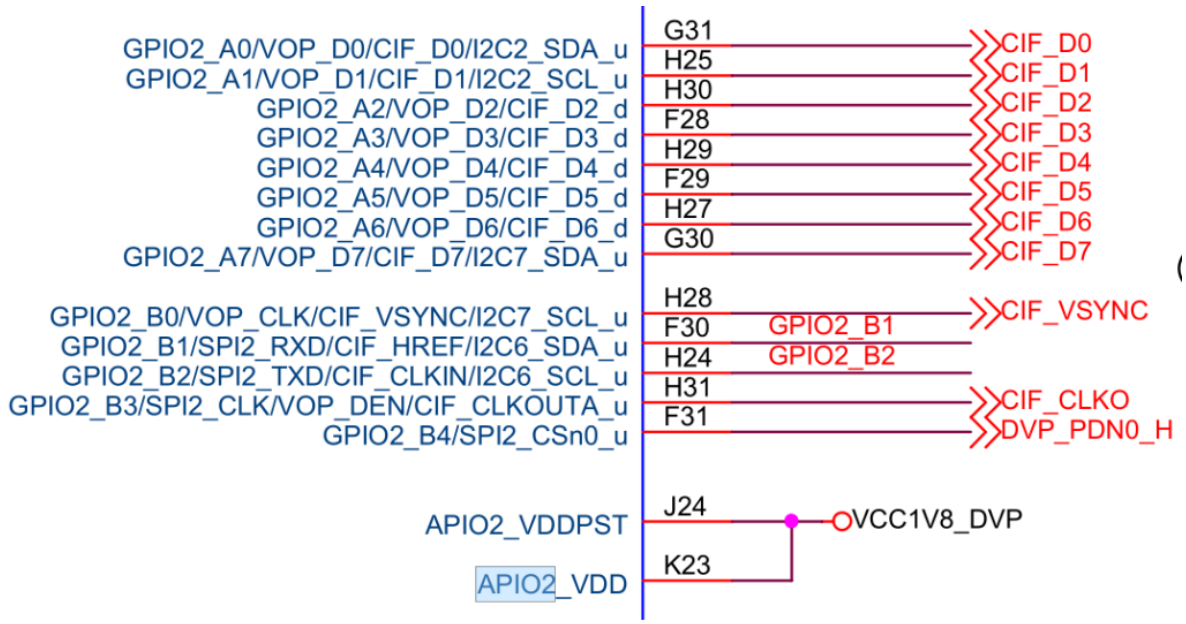
这里以RK3399为例，打开RK3399主控芯片对应的TRM芯片规格书中的GRM/PMUGRF章节搜索”VSEL”、”volsel”、”vsel”等关键字来找到io-domain寄存器。

GRF_IO_VSEL
Address: Operational Base + offset (0x0e640)

Bit	Attr	Reset Value	Description
31:16	RW	0x0000	write_enable bit0~15 write enable When bit 16=1, bit 0 can be written by software . When bit 16=0, bit 0 cannot be written by software; When bit 17=1, bit 1 can be written by software . When bit 17=0, bit 1 cannot be written by software; ..... When bit 31=1, bit 15 can be written by software . When bit 31=0, bit 15 cannot be written by software;
15:4	RO	0x0	reserved
3	RW	0x0	gpio1830_gpio4cd_ms 1'b0: IO domain works as 3.0V; 1'b1: IO domain works as 1.8V;
2	RW	0x0	sdmmc_gpio4b_ms 1'b0: IO domain works as 3.0V; 1'b1: IO domain works as 1.8V;
1	RW	0x0	audio_gpio3d4a_ms 1'b0: IO domain works as3.0V; 1'b1: IO domain works as 1.8V;
0	RW	0x0	bt656_gpio2ab_ms 1'b0: IO domain works as 3.0V; 1'b1: IO domain works as 1.8V;
9	RW	0x1	pmu1830_vol pmu IO 1.8v/3.0v select. 0: IO domain works as3.0v ; 1: IO domain works as 1.8v ;

通过关键字找到对应的寄存器。随后查看原理图上引脚对应的电源，比如，gpio2这路的整排引脚控制不正常：

Rockchip Confidential



这排引脚对应的电源是APIO2，接下来再去sdk代码的 kernel 目录下找到文档：kernel/Documentation/devicetree/bindings/power/rockchip-io-domain.txt。该文档有详细的描述软件中对应的APIO2这一路电源的定义：

```
Possible supplies for rk3399:
- bt656-supply:  The supply connected to APIO2_VDD.
- audio-supply:  The supply connected to APIO5_VDD.
- sdmmc-supply:  The supply connected to SDMMC0_VDD.
- gpio1830       The supply connected to APIO4_VDD.
Possible supplies for rk3399 pmu-domains:
- pmu1830-supply:The supply connected to PMUIO2_VDD.
```

在 dts 目录下搜索“io_domain”找到：

```
&io_domains {
    status = "okay";

    bt656-supply = <&vcc1v8_dvp >;
    audio-supply = <&vcca1v8_codec>;
    sdmmc-supply = <&vcc_sdio>;
    gpio1830-supply = <&vcc_3v0>;
};
```

确认软件配置里对应的电源设置跟硬件一样即可：

```
bt656-supply = <&vcc1v8_dvp>;
```

查看调试节点信息：

```
console:/ # cat sys/kernel/debug/regulator/regulator_summary
```

那么此处设置的1.8V跟硬件原理图一致才是正确，如果不一样就改成一致。

1.2 U-Boot配置

参考《RKDocs/rk3399/Rockchip_RK3399_Developer_Guide_Android7.1_Software_CN&EN.pdf》9.13节。==增加rkdevelop分支U-Boot配置io-domain功能==，该功能解决如下问题：

- 解决U-Boot下io-domain错误导致的IO电平不对或者iomux无法设置问题。
- 如果U-Boot阶段未设置io-domain，芯片会加载默认的配置，直到内核加载了DTS正确的配置，这个持续时间大概会有3~4s。如果某个io-domain连到3.0V，但是默认的配置为1.8V，则会出现电平不匹配的情况，有可能会损坏RK3399的IO。

配置好 Kernel 的io-domain，确认U-Boot和Kernel有下述两个提交：(u-boot/drivers/power/rockchip-io-domain.c)

```
u-boot:
commit 5a4e354a116b45b8ac2311aaa3464a7e88cd13d5
Author: Jianqun Xu <jay.xu@rock-chips.com>
Date: Thu Mar 19 15:38:19 2020 +0800
power: support rockchip io-domain set
This patch should work with kernel dtb node property - "uboot-set" in io-domain
node or pmu-io-domain node.
Change-Id: I86fa43d234091310ee08978ec42f5895fb010a8c
Signed-off-by: Jianqun Xu <jay.xu@rock-chips.com>
Signed-off-by: Wenping Zhang <wenping.zhang@rock-chips.com>
kernel:
commit f95ad65aabffa7450ff90a0dd4778cd5500de937
Author: Jianqun Xu <jay.xu@rock-chips.com>
Date: Fri Mar 20 09:52:57 2020 +0800
arm64: dts: rockchip: add 'uboot-set' in io-domain node for rk3399.
Add 'uboot-set' for io-domain node, which used by u-boot board init to set io
domain.
Change-Id: Ia897e240c04d0b362551952278dadc8b4f2a8f2f
Signed-off-by: Jianqun Xu <jay.xu@rock-chips.com>
Signed-off-by: Wenping Zhang <wenping.zhang@rock-chips.com>
```

2. Kernel中开发GPIO

本章节主要介绍Kernel开发GPIO的配置及方法。

2.1 GPIO复用

DTS配置

以RK3588 Android12平台举例，参考如下配置。如需操作gpio3b7引脚，DTS中配置`enable-gpios = <&gpio3 RK_PB7 GPIO_ACTIVE_HIGH>`。为了防止GPIO引脚在Kernel启动之前被复用为其他功能而导致GPIO不能控制，可以通过设置`pinctrl-names`和`pinctrl-0`属性将引脚复用为GPIO功能。如用户需要申请IO中断，参考gpio3b6可配置为施密特输入模式。

```
/{
    gpio_demo: gpio-demo {
        status = "okay";
        compatible = "rockchip,gpio-demo";
        enable-gpios = <&gpio3 RK_PB7 GPIO_ACTIVE_HIGH>;
        irq-gpios = <&gpio3 RK_PB6 GPIO_ACTIVE_HIGH>;
```

```
        pinctrl-names = "default";
        pinctrl-0 = <&gpio_pin &gpio_irq>;
    };
};

&pinctrl {
    gpio-pin {
        gpio_pin: gpio-pin {
            // Configure gpio3b7 iomux to GPIO
            rockchip,pins = <3 RK_PB7 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };

    gpio-irq {
        gpio_irq: gpio-irq {
            // Configure gpio3b6 iomux to GPIO and schmidt input mode
            rockchip,pins = <3 RK_PB6 RK_FUNC_GPIO &pcfg_pull_none_smt>;
        };
    };
};
```

`pinctrl-0`引用的属性节点遵循如下规则：

- 必须在pinctrl节点下；
- 必须以function+group的形式添加；
- 遵循其他DTS的基本规则；
- function+group的格式如下:

```
function {
    group {
        rockchip,pin = <gpiobank gpionum iomuxfunc &config>;
    };
};
```

`iomuxfunc`是指复用功能，`RK_FUNC_GPIO`是配置复用为GPIO功能。`config`为配置IO引脚上拉、下拉、端口施密特触发模式和驱动强度。参考SDK文件rockchip-pinconf.dtsi，RK在DTS的pinctrl节点下列举了所有配置。如下：

```
&pinctrl {
    pcfg_pull_up: pcfg-pull-up {
        bias-pull-up;
    };

    pcfg_pull_down: pcfg-pull-down {
        bias-pull-down;
    };

    pcfg_pull_none: pcfg-pull-none {
        bias-disable;
    };

    pcfg_pull_none_smt: pcfg-pull-none-smt {
        bias-disable;
        input-schmitt-enable;
    };
```

```
    pcfg_pull_none_drv_level_0: pcfg-pull-none-drv-level-0 {
        bias-disable;
        drive-strength = <0>;
    };

    ......
    pcfg_output_high: pcfg-output-high {
        output-high;
    };

    ......
    pcfg_output_low: pcfg-output-low {
        output-low;
    };

    ......
};
```

其中`drive-strength = <2>`表示驱动强度等级为2，写入驱动强度寄存器中值即为2。以RK3588举例，TRM中gpio2a0的具体驱动强度等级如下图所示，驱动强度等级2代表50ohm。

用户驱动代码

以RK3588 Android12平台举例，DTS配置完成后，用户驱动代码获取DTS配置操作GPIO。用户驱动代码可通过如下函数申请和操作GPIO。

```
// Get the GPIO device. If multiple devices are used, the index parameter must
be included
struct gpio_desc *gpiod_get(struct device *dev, const char *con_id,
                    enum gpiod_flags flags)

struct gpio_desc *gpiod_get_index(struct device *dev,
                      const char *con_id, unsigned int idx,
                      enum gpiod_flags flags)
// Release the GPIO device
void gpiod_put(struct gpio_desc *desc)
void gpiod_put_array(struct gpio_descs *descs)

// Set gpio input/output
int gpiod_direction_input(struct gpio_desc *desc)
int gpiod_direction_output(struct gpio_desc *desc, int value)

// Get/Set gpio value
int gpiod_get_value(const struct gpio_desc *desc);
void gpiod_set_value(struct gpio_desc *desc, int value);

//Get the GPIO IRQ interrupt number
int gpiod_to_irq(const struct gpio_desc *desc)
......
```

用户驱动操作GPIO代码参考如下：

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (c) 2020 Rockchip Electronics Co. Ltd.
 *
 * Author:Johnny Shi <nengqiang.shi@rock-chips.com>
```

```
 */

/* The following code is based on the RK3588 Android 12 platform */
#include <linux/gpio/consumer.h>
#include <linux/interrupt.h>
#include <linux/kernel.h>
#include <linux/math64.h>
#include <linux/module.h>
#include <linux/of_graph.h>
#include <linux/phy/phy.h>
#include <linux/platform_device.h>

static irqreturn_t irq_demo_handler(int irq, void *dev_id)
{
    printk("------irq_demo------\n");

    return IRQ_HANDLED;
}

static int gpio_demo_probe(struct platform_device *pdev)
{
    struct gpio_desc *enable_gpio, *irq_gpio;
    int irq;
    int ret;

    printk("------gpio_demo_probe------\n");

    // Get the gpio3b7 pin and set it as output
    enable_gpio = devm_gpiod_get_index(&pdev->dev, "enable", 0, GPIOD_OUT_HIGH);
    if (IS_ERR(enable_gpio)) {
        ret = PTR_ERR(enable_gpio);
         dev_err(&pdev->dev, "failed to request enable GPIO: %d\n", ret);
        return ret;
    }

    // Get the gpio3b6 pin and set it as input
    irq_gpio = devm_gpiod_get_index(&pdev->dev, "irq", 0, GPIOD_IN);
    if (IS_ERR(irq_gpio)) {
        ret = PTR_ERR(irq_gpio);
         dev_err(&pdev->dev, "failed to request irq GPIO: %d\n", ret);
        return ret;
    }
    // Get the gpio3b6 IRQ interrupt number and request IRQ
    irq = gpiod_to_irq(irq_gpio);
    ret = request_irq(irq, irq_demo_handler, IRQF_TRIGGER_FALLING, "demo_irq",
NULL);
    if (ret) {
         dev_err(&pdev->dev, "failed to request IRQ: %d\n", ret);
        return ret;
    }

    // Set gpio3b7 pin output high
    gpiod_set_value(enable_gpio, 1);

    return 0;
}

static const struct of_device_id gpio_demo_of_match[] = {
```

```
    { .compatible = "rockchip,gpio-demo" },
    {}
};

static struct platform_driver gpio_demo_driver = {
    .driver = {
        .name = "gpio-demo",
        .of_match_table = of_match_ptr(gpio_demo_of_match),
    },
    .probe  = gpio_demo_probe,
//    .remove = gpio_demo_remove,
};
module_platform_driver(gpio_demo_driver);

MODULE_DESCRIPTION("Rockchip gpio demo driver");
MODULE_LICENSE("GPL v2");
```

依据DTS配置，用户驱动通过`devm_gpiod_get_index(&pdev->dev, "enable", 0, GPIOD_OUT_HIGH)`获取gpio3b7引脚，依据DTS中的属性配置`GPIO_ACTIVE_HIGH`将引脚设置为高电平有效，`GPIOD_OUT_HIGH`则将gpio3b7初始化为输出。`gpiod_set_value(enable_gpio, 1)`使得gpio3b7输出高电平。参考上述代码，用户申请IRQ需先通过`devm_gpiod_get_index(&pdev->dev, "irq", 0, GPIOD_IN)`获取gpio3b6引脚，并初始化为输入。通过`gpiod_to_irq(irq_gpio)`，`request_irq(irq, irq_demo_handler, IRQF_TRIGGER_FALLING, "demo_irq", NULL)`申请IRQ。

2.2 GPIO输入输出

DTS中通过配置pinctrl-names = "default"，然后在pinctrl-0中配置GPIO属性为pcfg_output_high或者pcfg_output_low，让GPIO直接输出高低电平。因为驱动在加载的时候，会自动配置pinctrl-0中的GPIO属性。

例如：

```
imx415: imx415@1a {
    compatible = "sony,imx415";
    reg = <0x1a>;
    clocks = <&cru CLK_MIPI_CAMARAOUT_M1>;
    clock-names = "xvclk";
    pinctrl-names = "default";
-   pinctrl-0 = <&mipim0_camera1_clk>;
+   pinctrl-0 = <&mipim0_camera1_clk &test_gpio>; //添加一个test_gpio
    power-domains = <&power RK3588_PD_VI>;
    ......
}

&pinctrl {
    test {
        test_gpio: test-gpio {
            //GPIO3_B1配置为高电平输出，也可以配置为pcfg_output_low，低电平输出。
            rockchip,pins = <3 RK_PB1 RK_FUNC_GPIO &pcfg_output_high>;
        };
    };
}
```

GPIO输入配置，一般设置为pcfg_pull_none或者pcfg_pull_none_smt即可。

2.3 动态设置IO复用

用户可以实现动态切换IO引脚复用，如内核运行时想要某阶段IO引脚当作UART使用，另一阶段当作普通GPIO口使用。以RK3588 Android12平台举例，配置如下。

DTS配置

`pinctrl-names` 除了内核中定义的四种状态，用户还可以自定义状态。`"default"` 状态默认设置uart3引脚，用户驱动匹配后内核会自行绑定设置，用户驱动无需再进行设置。用户自定义状态引脚参考 `pinctrl-names= "iomux-exchange"` 设置，用户可在用户驱动中解析并按需进行动态设置引脚复用。

```
/{
    iomux_exchange: iomux-exchange {
        compatible= "iomux-exchange-demo";
        pinctrl-names= "default", "iomux-exchange";
        pinctrl-0= <&uart3m0_xfer>;
        pinctrl-1= <&uart3m0_gpio>;
    };

    &pinctrl {
        uart3 {
            /omit-if-no-ref/
            uart3m0_xfer: uart3m0-xfer {
                rockchip,pins =
                    /* uart3_rx_m0 */
                    <1 RK_PC0 10 &pcfg_pull_up>,
                    /* uart3_tx_m0 */
                    <1 RK_PC1 10 &pcfg_pull_up>;
            };
        uart3-gpio{
            uart3m0_gpio: uart3m0-gpio {
                rockchip,pins =
                    <1 RK_PC0 RK_FUNC_GPIO &pcfg_pull_none>,
                    <1 RK_PC1 RK_FUNC_GPIO &pcfg_pull_none>;
            };
        };
    };
};
```

用户驱动代码

用户驱动中通过`devm_pinctrl_get`获取pinctrl资源，`pinctrl_lookup_state`获取用户自定义状态的GPIO配置，`pinctrl_select_state`设置IO引脚状态。用户驱动代码参考如下：

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (c) 2020 Rockchip Electronics Co. Ltd.
 *
 * Author:Johnny Shi <nengqiang.shi@rock-chips.com>
 */
/* The following code is based on the RK3588 Android 12 platform */
#include <linux/gpio/consumer.h>
#include <linux/interrupt.h>
#include <linux/kernel.h>
#include <linux/math64.h>
#include <linux/module.h>
```

```c
#include <linux/of_graph.h>
#include <linux/phy/phy.h>
#include <linux/platform_device.h>
#include <linux/pinctrl/consumer.h>

struct pinctrl *uart_pinctrl;
struct pinctrl_state   *uart_default;//uart
struct pinctrl_state   *uart_gpio;//gpio

static int iomux_exchange(int select) {
    int ret = 0;
    if (select) {
        ret = pinctrl_select_state(uart_pinctrl, uart_gpio);
    } else {
        ret = pinctrl_select_state(uart_pinctrl, uart_default);
    }

    return ret;
}

static int iomux_exchange_demo_probe(struct platform_device *pdev)
{
    printk("------iomux_exchange_demo_probe------\n");
    uart_pinctrl = devm_pinctrl_get(&pdev->dev);
    if (IS_ERR_OR_NULL(uart_pinctrl)) {
        printk("Failed to get pin ctrl\n");
        return PTR_ERR(uart_pinctrl);
    }

    // Gets the pinctrl status property of "default"
    uart_default = pinctrl_lookup_state(uart_pinctrl, "default");
    if (IS_ERR_OR_NULL(uart_default)) {
        printk("Failed to lookup pinctrl default state\n");
        return PTR_ERR(uart_default);
    }

    // Get the custom "iomux-exchange" pinctrl status properties
    uart_gpio = pinctrl_lookup_state(uart_pinctrl, "iomux-exchange");
    if (IS_ERR_OR_NULL(uart_gpio)) {
        printk("Failed to lookup pinctrl iomux-exchange state\n");
        return PTR_ERR(uart_gpio);
    }

    iomux_exchange(1);
    //iomux_exchange(0);
    return 0;
}

static const struct of_device_id iomux_exchange_demo_of_match[] = {
    { .compatible = "iomux-exchange-demo" },
    {}
};

static struct platform_driver iomux_exchange_demo_driver = {
    .driver = {
        .name = "iomux-exchange-demo",
        .of_match_table = of_match_ptr(iomux_exchange_demo_of_match),
    },
```

```
    .probe  = iomux_exchange_demo_probe,
//    .remove = iomux_exchange_demo_remove,
};
module_platform_driver(iomux_exchange_demo_driver);

MODULE_DESCRIPTION("Rockchip iomux exchange demo driver");
MODULE_LICENSE("GPL v2");
```

上述代码中若不调用`iomux_exchange(1)`，引脚会默认配置为UART引脚。系统启动后，终端输入命令查看IO复用：

```
console:/ # cat sys/kernel/debug/pinctrl/pinctrl-rockchip-pinctrl/pinmux-pins
......
pin 44 (gpio1-12): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 45 (gpio1-13): fde60000.dp gpio1:45 function dp group dp1-hpd
pin 46 (gpio1-14): 5-001a (GPIO UNCLAIMED) function mipi group mipim0-camera1-
clk
pin 47 (gpio1-15): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 48 (gpio1-16): iomux-exchange (GPIO UNCLAIMED) function uart3 group uart3m0-
xfer
pin 49 (gpio1-17): iomux-exchange (GPIO UNCLAIMED) function uart3 group uart3m0-
xfer
pin 50 (gpio1-18): 7-0011 (GPIO UNCLAIMED) function i2s0 group i2s0-mclk
pin 51 (gpio1-19): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
pin 52 (gpio1-20): (MUX UNCLAIMED) gpio1:52
pin 53 (gpio1-21): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
......
```

可以看到gpio1-16和gpio1-17被设置为UART引脚。当调用`iomux_exchange(1)`后，输入命令查看：

```
console:/ # cat sys/kernel/debug/pinctrl/pinctrl-rockchip-pinctrl/pinmux-pins
......
pin 44 (gpio1-12): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 45 (gpio1-13): fde60000.dp gpio1:45 function dp group dp1-hpd
pin 46 (gpio1-14): 5-001a (GPIO UNCLAIMED) function mipi group mipim0-camera1-
clk
pin 47 (gpio1-15): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 48 (gpio1-16): iomux-exchange (GPIO UNCLAIMED) function uart3-gpio group
uart3m0-gpio
pin 49 (gpio1-17): iomux-exchange (GPIO UNCLAIMED) function uart3-gpio group
uart3m0-gpio
pin 50 (gpio1-18): 7-0011 (GPIO UNCLAIMED) function i2s0 group i2s0-mclk
pin 51 (gpio1-19): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
pin 52 (gpio1-20): (MUX UNCLAIMED) gpio1:52
pin 53 (gpio1-21): fe470000.i2s (GPIO UNCLAIMED) function i2s0 group i2s0-idle
......
```

可以看到gpio1-16和gpio1-17被成功复用为GPIO功能。

3. U-Boot中开发GPIO

用户有在U-Boot中操作GPIO的需求，可以通过`readl`和`writel`直接读写寄存器地址来操作GPIO。但是该方法代码不具备可移植性，这里提供一种通过U-Boot驱动来操作GPIO的方法。以RK3588 Android12平台举例，具体U-Boot DTS配置和驱动代码参考如下。

3.1 U-Boot LED驱动开发

DTS配置

U-Boot有自己的DTS文件，编译时会自动生成相应的DTB文件，被添加在u-boot.bin末尾。U-Boot开发GPIO的DTS配置与Kernel相同，不过需要在节点中加`u-boot,dm-spl`或`u-boot,dm-pre-reloc`属性。

```
/{
    leds {
        compatible = "gpio-leds";
        status = "okay";
        u-boot,dm-spl;

        blue-led {
            u-boot,dm-spl;
            gpios = <&gpio2 RK_PA1 GPIO_ACTIVE_LOW>;
            label = "blue_full";
            default-state = "on";
        };

        green-led {
            u-boot,dm-spl;
            gpios = <&gpio2 RK_PA0 GPIO_ACTIVE_LOW>;
            label = "green_led";
            default-state = "on";
        };
    };
};
```

注意：DTS文件中“gpio-leds”的配置一般放在Kernel中，“label”配置选项需要添加，不然led_gpio.c驱动会bind出错。如下补丁可以打印“label”配置选项：

```
diff --git a/drivers/led/led_gpio.c b/drivers/led/led_gpio.c
index 56702a1417..18062f2ac1 100644
--- a/drivers/led/led_gpio.c
+++ b/drivers/led/led_gpio.c
@@ -112,6 +112,7 @@ static int led_gpio_bind(struct udevice *parent)
                const char *label;

                label = ofnode_read_string(node, "label");
+               printf("%s: label %s\n", __func__, label);
                if (!label) {
                        debug("%s: node %s has no label\n", __func__,
                              ofnode_get_name(node));
```

用户编译完U-Boot后可通过`fdtdump`命令检查DTB内容：

```
fdtdump ./u-boot.dtb | less
```

用户驱动代码

这里主要介绍下U-Boot中LED的驱动。首先打开配置：

```
/u-boot/configs/rk3588_defconfig
CONFIG_LED=y
CONFIG_LED_GPIO=y
```

具体框架代码参考如下：

```
u-boot/drivers/led/led-uclass.c // 默认编译
u-boot/drivers/led/led_gpio.c // 支持 compatible = "gpio-leds"
```

相关代码接口

```
// 获取led device
int led_get_by_label(const char *label, struct udevice **devp);
// 设置/获取led状态
int led_set_state(struct udevice *dev, enum led_state_t state);
enum led_state_t led_get_state(struct udevice *dev);
// 忽略，目前未做底层驱动实现
int led_set_period(struct udevice *dev, int period_ms);
```

用户只需要调用`led_get_by_label`即可实现LED亮灯。参考代码如下，将`led_get_by_label`添加到`board_late_init`执行，系统启动后可观察到gpio2a0和gpio2a1引脚的LED亮灯。

```
/u-boot/arch/arm/mach-rockchip/board.c
#include <led.h>

int board_late_init(void)
{
    struct udevice *led_dev;
    ...
    led_get_by_label("blue_full", &led_dev);
    led_set_state(led_dev, LEDST_ON);
    ...
}
```

U-Boot与Kernel驱动不同，虽然引入了device-driver开发模型，但初始化阶段不会像Kernel那样自动发起已注册device-driver的probe。driver的probe必须由用户主动调用发起 。即用户主动调用`led_get_by_label`后，才会执行LED驱动probe函数`led_gpio_probe`。`led_get_by_label`的核心调用是通过`device_probe`执行到LED驱动的probe。并且可以通过`led_set_state`设置LED灯亮灭状态。

参考LED驱动代码，用户可以自行设计U-Boot的GPIO驱动。获取用户设备驱动接口如下：

```
int uclass_get_device(enum uclass_id id, int index, struct udevice **devp);
int uclass_get_device_by_name(enum uclass_id id, const char *name,
struct udevice **devp);
int uclass_get_device_by_seq(enum uclass_id id, int seq, struct udevice **devp);
int uclass_get_device_by_of_offset(enum uclass_id id, int node, struct udevice
**devp);
int uclass_get_device_by_ofnode(enum uclass_id id, ofnode node, struct udevice
**devp);
int uclass_get_device_by_phandle_id(enum uclass_id id,
```

```
int phandle_id, struct udevice **devp);
int uclass_get_device_by_phandle(enum uclass_id id,
struct udevice *parent, struct udevice **devp);
int uclass_get_device_by_driver(enum uclass_id id,
const struct driver *drv, struct udevice
**devp);
int uclass_get_device_tail(struct udevice *dev, int ret, struct udevice **devp);
......
```

操作GPIO接口代码如下：

```
// 申请/释放GPIO
int gpio_request_by_name(struct udevice *dev, const char *list_name,
int index, struct gpio_desc *desc, int flags);
int gpio_request_by_name_nodev(ofnode node, const char *list_name, int index,
struct gpio_desc *desc, int flags);
int gpio_request_list_by_name(struct udevice *dev, const char *list_name,
struct gpio_desc *desc_list, int max_count, int
flags);
int gpio_request_list_by_name_nodev(ofnode node, const char *list_name,
struct gpio_desc *desc_list, int max_count,
int flags);
int dm_gpio_free(struct udevice *dev, struct gpio_desc *desc)
// 配置GPIO方向。@flags: GPIOD_IS_OUT（输出）和 GPIOD_IS_IN（输入）
int dm_gpio_set_dir_flags(struct gpio_desc *desc, ulong flags);
// 设置/获取GPIO电平
int dm_gpio_get_value(const struct gpio_desc *desc)
int dm_gpio_set_value(const struct gpio_desc *desc, int value)
```

`udevice`通过`uclass_id`获取，参考LED驱动注册如下：

```
U_BOOT_DRIVER(led_gpio) = {
  .name  = "gpio_led",
  .id = UCLASS_LED,
  .of_match = led_gpio_ids,
  .ops   = &gpio_led_ops,
  .priv_auto_alloc_size = sizeof(struct led_gpio_priv),
  .bind  = led_gpio_bind,
  .probe  = led_gpio_probe,
  .remove = led_gpio_remove,
};
```

`UCLASS_LED`定义在u-boot/include/dm/uclass-id.h。

3.2 U-Boot GPIO驱动开发

上述LED驱动在U-Boot驱动中已经写好，用户只需配置DTS和调用驱动接口即可。下面介绍下U-Boot GPIO的驱动开发demo，打开如下配置：

```
/u-boot/configs/rk3588_defconfig
CONFIG_DM_GPIO=y
CONFIG_GPIO_HOG=y
CONFIG_ARCH_ROCKCHIP=y
CONFIG_SPL_GPIO_SUPPORT=y
```

DTS配置

以操作gpio3b7为例，U-Boot DTS配置如下：

```
gpio_uboot_demo {
    u-boot,dm-pre-reloc;
    compatible="gpio_uboot_demo";
    enable-gpios = <&gpio3 RK_PB7 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

&gpio3 {
    u-boot,dm-pre-reloc;
    status = "okay";
};
```

用户驱动代码

```
/*
 * Copyright (c) 2015 Google, Inc
 * Written by Simon Glass <sjg@chromium.org>
 *
 * SPDX-License-Identifier:    GPL-2.0+
 */

/* The following code is based on the RK3588 Android 12 platform */
#include <common.h>
#include <dm.h>
#include <errno.h>
#include <asm/gpio.h>
#include <dm/lists.h>
#include <dm/ofnode.h>

static int gpio_uboot_demo_probe(struct udevice *dev)
{
    struct gpio_desc gpio_uboot ;
    int ret;

    printf("----gpio_uboot_demo_probe----\n");
    ret = gpio_request_by_name(dev, "enable-gpios", 0, &gpio_uboot,
GPIOD_IS_OUT);
    if(ret)
    {
        printf("can not get eepro_wp gpio\n");
    }

    dm_gpio_set_value(&gpio_uboot, 1);
    return 0;
}
```

```
static int gpio_uboot_demo_remove(struct udevice *dev)
{
    printf("----gpio_uboot_demo_remove----\n");
    return 0;
}

static const struct udevice_id gpio_uboot_demo_ids[] = {
    { .compatible = "gpio_uboot_demo" },
    { }
};

U_BOOT_DRIVER(gpio_uboot_demo) = {
    .name         = "gpio_uboot_demo",
    .id             = UCLASS_GPIO,
    .of_match     = gpio_uboot_demo_ids,
    .probe         = gpio_uboot_demo_probe,
    .remove         = gpio_uboot_demo_remove,
};
```

用户驱动通过`gpio_request_by_name(dev, "enable-gpios", 0, &gpio_uboot, GPIOD_IS_OUT)`获取gpio3b7引脚，并设置为输出。GPIO probe函数需要用户主动调用相关接口执行，如下：

```
/u-boot/arch/arm/mach-rockchip/board.c
int board_late_init(void)
{
    struct udevice *dev;
    ...
    uclass_get_device_by_driver(UCLASS_GPIO, DM_GET_DRIVER(gpio_uboot_demo),
&dev);
    ...
}
```

在`board_late_init`中调用`uclass_get_device_by_driver`最终会调用上述驱动的probe，`dm_gpio_set_value(&gpio_uboot, 1)`实现输出高电平。

4. 用户态开发GPIO

用户态开发GPIO的形式有很多种，这里以常见的echo命令举例说明。Linux内核提供了一套在用户态调试GPIO的接口，在`/sys/class/gpio/`目录下：

```
console:/sys/class/gpio# ls
export     gpiochip128  gpiochip509  gpiochip96
gpiochip0  gpiochip32   gpiochip64   unexport
```

`export`文件用于通知系统导出需要控制的GPIO引脚编号。`unexport`用于通知系统取消导出。`gpiochipN`保存系统中GPIO寄存器的信息，包括每个寄存器控制引脚的起始编号base，寄存器名称，引脚总数。

举例导出引脚gpio1b1来操作。首先计算此引脚编号，引脚编号 = 控制引脚的寄存器基数 + 控制引脚寄存器位数。gpio1b1= 32 + 9 =41。导出gpio1b1：

```
console:/sys/class/gpio# echo 41 > export
```

设置输入输出，"in"为输入，“out”为输出：

```
console:/sys/class/gpio/gpio41# echo out > direction
```

查看方向：

```
console:/sys/class/gpio/gpio41# cat direction
```

设置输出：

```
console:/sys/class/gpio/gpio41# echo 1 > value
```

查看输入输出：

```
console:/sys/class/gpio/gpio41# cat value
```

取消导出：

```
console:/sys/class/gpio# echo 41 > unexport
```

上述操作需要确保IO复用为GPIO功能，才能做正常的GPIO操作。如果复用为其他功能则无法操作该GPIO输出输入。需要提前排查IO复用情况，除了用io工具查看寄存器看IO复用和配置，还可以下面的命令来查看所有的引脚复用和配置情况：

```
console:/ # cat sys/kernel/debug/pinctrl/pinctrl-rockchip-pinctrl/pinmux-pins
Pinmux settings per pin
Format: pin (name): mux_owner gpio_owner hog?
pin 0 (gpio0-0): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 1 (gpio0-1): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 2 (gpio0-2): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 3 (gpio0-3): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 4 (gpio0-4): (MUX UNCLAIMED) gpio0:4
pin 5 (gpio0-5): (MUX UNCLAIMED) gpio0:5
pin 6 (gpio0-6): ff420030.pwm (GPIO UNCLAIMED) function pwm3a group pwm3a-pin
pin 7 (gpio0-7): fe320000.dwmmc (GPIO UNCLAIMED) function sdmmc group sdmcc-cd
pin 8 (gpio0-8): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 9 (gpio0-9): (MUX UNCLAIMED) gpio0:9
pin 10 (gpio0-10): (MUX UNCLAIMED) gpio0:10
......

console:/ # cat sys/kernel/debug/gpio
gpiochip0: GPIOs 0-31, parent: platform/pinctrl, gpio0:
 gpio-1   (                    |vcc3v0_sd           ) out hi
 gpio-4   (                    |bt_default_wake_host) in  lo
 gpio-5   (                    |GPIO Key Power      ) in  hi
 gpio-9   (                    |bt_default_reset    ) out lo
 gpio-10  (                    |reset               ) out hi
 gpio-11  (                    |spk-con             ) out lo

gpiochip1: GPIOs 32-63, parent: platform/pinctrl, gpio1:
 gpio-34  (                    |int-n               ) in  hi
```

```
 gpio-45  (                    |enable              ) out hi

gpiochip2: GPIOs 64-95, parent: platform/pinctrl, gpio2:
 gpio-64  (                    |vbus-5v             ) out lo
 gpio-83  (                    |bt_default_rts      ) in  hi
 gpio-90  (                    |bt_default_wake     ) in  lo

gpiochip3: GPIOs 96-127, parent: platform/pinctrl, gpio3:
 gpio-111 (                    |mdio-reset          ) out hi

gpiochip4: GPIOs 128-159, parent: platform/pinctrl, gpio4:
 gpio-150 (                    |?                   ) out hi
 gpio-153 (                    |vcc5v0_host         ) out hi
 gpio-157 (                    |enable              ) out hi
 gpio-158 (                    |vcc_lcd             ) out hi
```

5. IO休眠唤醒设置

现在芯片级的休眠唤醒操作主要在ATF中，这部分代码是没有开放的，为满足不同的产品需求，可以通过DTS配置系统sleep时进入不同的低功耗模式。不同的项目对唤醒源的需求不同，在DTS中可以配置对应的唤醒源使能。

系统待机流程一般会有如下操作：关闭 power domain、模块 IP、时钟、PLL、DDR进入自刷新、系统总线切到低速时钟（24M 或 32K）、vdd_arm /vdd_log断电、配置唤醒源等。为了满足不同产品对待机模式的需求，目前都是通过DTS 节点把相关配置在开机阶段传递给trust。以RK3588平台为例：

```
arch/arm64/boot/dts/rockchip/rk3588s.dtsi
rockchip_suspend: rockchip-suspend {
        compatible = "rockchip,pm-rk3588";
        status = "disabled";
        // Set the sleep log switch. 0: printing is disabled, 1: printing is
enabled
        rockchip,sleep-debug-en = <0>;
         // Sleep configuration
        rockchip,sleep-mode-config = <
                (0
                | RKPM_SLP_ARMOFF_LOGOFF //进入深度休眠断电vdd_arm和vdd_log
                | RKPM_SLP_PMU_PMUALIVE_32K //进入深度休眠时使用32K时钟源作为系统时钟
                | RKPM_SLP_PMU_DIS_OSC //关闭24M晶振，最低功耗模式时可使能
                | RKPM_SLP_32K_EXT //休眠时的32K时钟源是否选用外部的32K钟源
                )
        >;
        // Wakeup source configuration
        rockchip,wakeup-config = <
                (0
                | RKPM_GPIO_WKUP_EN //GPIO0唤醒
                )
        >;
};
```

5.1 配置GPIO唤醒

以RK3588为例，GPIO0组的引脚电源休眠时默认配置打开，所以用户不需要再配置。如果需要配置GPIO0外的其他GPIO作为唤醒源，需要做以下配置：

- 配置这路GPIO休眠的时候不要断电

```
vdd_log_s0: DCDC_REG3 {
    regulator-always-on;
    regulator-boot-on;
    regulator-min-microvolt = <675000>;
    regulator-max-microvolt = <750000>;
    regulator-ramp-delay = <12500>;
    regulator-name = "vdd_log_s0";
    regulator-state-mem {
    // regulator-off-in-suspend;
    regulator-on-in-suspend;
    regulator-suspend-microvolt = <750000>;
    };
};
```

`regulator-off-in-suspend`表示休眠的时候这路电源要关闭，如果休眠要打开需要改为`regulator-on-in-suspend`。

- 配置rockchip_suspend节点

```
&rockchip_suspend {
        status = "okay";
        rockchip,sleep-debug-en = <1>;
        rockchip,sleep-mode-config = <
                (0
                 // | RKPM_SLP_ARMOFF_LOGOFF
                | RKPM_SLP_ARMOFF_DDRPD
                | RKPM_SLP_PMU_PMUALIVE_32K
                | RKPM_SLP_PMU_DIS_OSC
                | RKPM_SLP_32K_EXT
                )
        >;

        rockchip,wakeup-config = <
                (0
                | RKPM_GPIO_WKUP_EN
                | RKPM_CPU0_WKUP_EN
                )
        >;
};
```

rockchip_suspend节点中`RKPM_SLP_ARMOFF_LOGOFF`表示断电vdd_arm和vdd_log，改为`RKPM_SLP_ARMOFF_DDRPD`不断电vdd_log。rockchip_suspend中还需要配置`RKPM_CPU0_WKUP_EN`。

注意不是所有的GPIO都可以作为唤醒源，具体请查看下TRM文档 PMU_WAKEUP_INT_CON 寄存器，有很多平台只支持 GPIO0 这组的 IO 口为唤醒源。

5.2 配置休眠时GPIO电平

以RK3399平台举例：

```
&rockchip_suspend {
    status = "okay";
+    rockchip,power-ctrl =
+      <&gpio1 17 GPIO_ACTIVE_HIGH>, //配置休眠时GPIO1_C1为高电平
+      <&gpio1 14 GPIO_ACTIVE_HIGH>; //配置休眠时GPIO1_B6为高电平
```

rockchip,power-ctrl 只能选择 PMUIO1/2上的GPIO，最好是把这个io放到gpio0上面，否则休眠后io状态不能保持。新的休眠框架会把休眠时非pmu控制的电源域io恢复到默认状态。

6. GPIO扩展芯片

本章介绍GPIO的扩展芯片的基本配置和调用方式。如下以PCA9555示例说明（PCA9555是一款16位通用输入/输出GPIO扩展器，具有中断和弱上拉电阻，适用于I2c总线/SMBus应用）。

DTS配置

```
&i2c6 {
    status = "okay";

    pca9555_20: pca9555_20@20 {
        compatible = "nxp,pca9555";
        reg = <0x20>;
    };

    imx586: imx586@1a {
        compatible = "sony,imx586";
        status = "okay";
        reset-gpio-index = <503>;
        ......
    };
};
```

reset-gpio-index为gpio的编号，可以通过命令 cat /sys/kernel/debug/gpio确认。

驱动调用

```
ret |= of_property_read_u32(node, "reset-gpio-index",
        &test->reset_gpio_index);

gpio_to_desc(test->reset_gpio_index);
```

7. GPIO模拟I2C

本章介绍GPIO的模拟I2C的基本配置。对应驱动链路为：kernel/drivers/i2c/busses/i2c-gpio.c

config配置

在defconfig中，设置CONFIG_I2C_GPIO=y，或者menuconfig中选择GPIO-based bitbanging I2C

对应的选项为：

```
Device Drivers->
    I2C support  --->
        I2C Hardware Bus support  --->
            <*> GPIO-based bitbanging I2C
```

dts配置

```
i2c9: i2c@9 {
    compatible = "i2c-gpio";
    pinctrl-names = "default";
    pinctrl-0 = <&i2c9_xfer>;
    scl-gpios = <&gpio3 RK_PB3 GPIO_ACTIVE_HIGH>; //scl
    sda-gpios = <&gpio3 RK_PB4 GPIO_ACTIVE_HIGH>; //sda
    i2c-gpio,delay-us = <2>;    /* ~100 kHz */
    #address-cells = <1>;
    #size-cells = <0>;
};

&pinctrl {
    i2c9 {
        /omit-if-no-ref/
        i2c9_xfer: i2c9-xfer {
            rockchip,pins =
                /* i2c9_scl */
                <3 RK_PB3 0 &pcfg_pull_none_smt>,
                /* i2c9_sda */
                <3 RK_PB4 0 &pcfg_pull_none_smt>;
        };
    };
};
```

8. GPIO Debug工具

本章介绍GPIO的debug工具。

8.1 IO工具

Rockchip平台提供了IO工具，可用于直接读取和设置寄存器的值，方便用户debug。

8.1.1 Kernel阶段IO工具

Android10及以上版本因安全标准因素，调试时需手动配置。举例RK3399 Android12平台，使用IO指令需打开如下配置：

```
kernel-5.10/kernel/configs/android-11.config
CONFIG_DEVMEM=y
```

IO工具所包含的命令参数详解可以在串口终端或者 adb 输入“io” 回车后便可罗列出。

```
console:/# io
Raw memory i/o utility - $Revision: 1.5 $

io -v -1|2|4 -r|w [-l <len>] [-f <file>] <addr> [<value>]

    -v          Verbose, asks for confirmation
    -1|2|4      Sets memory access size in bytes (default byte)
    -l <len>    Length in bytes of area to access (defaults to
                one access, or whole file length)
    -r|w        Read from or Write to memory (default read)
    -f <file>   File to write on memory read, or
                to read on memory write
    <addr>      The memory address to access
    <val>       The value to write (implies -w)

Examples:
    io 0x1000                   Reads one byte from 0x1000
    io 0x1000 0x12              Writes 0x12 to location 0x1000
    io -2 -l 8 0x1000           Reads 8 words from 0x1000
    io -r -f dmp -l 100 200     Reads 100 bytes from addr 200 to file
    io -w -f img 0x10000        Writes the whole of file to memory

Note access size (-1|2|4) does not apply to file based accesses.
```

IO工具查询GPIO复用

以下用RK3399平台来举例，用户可通过RK3399主控芯片详细规格书TRM查询寄存器来确定是否存在IO复用。例如想查询的是gpio4b0引脚，在原理图上有以下复用状态：

```
GPIO4_B0/SDMMC0_D0/UART2A_RX_u
```

通过在RK3399 TRM上搜索gpio4b0，找到寄存器描述如下：

通过TRM文档确定该寄存器的基地址名称是“GRF”，偏移0x0e024：

通过在datasheet中搜索address mapping，并在其中找到GRF的基地址：

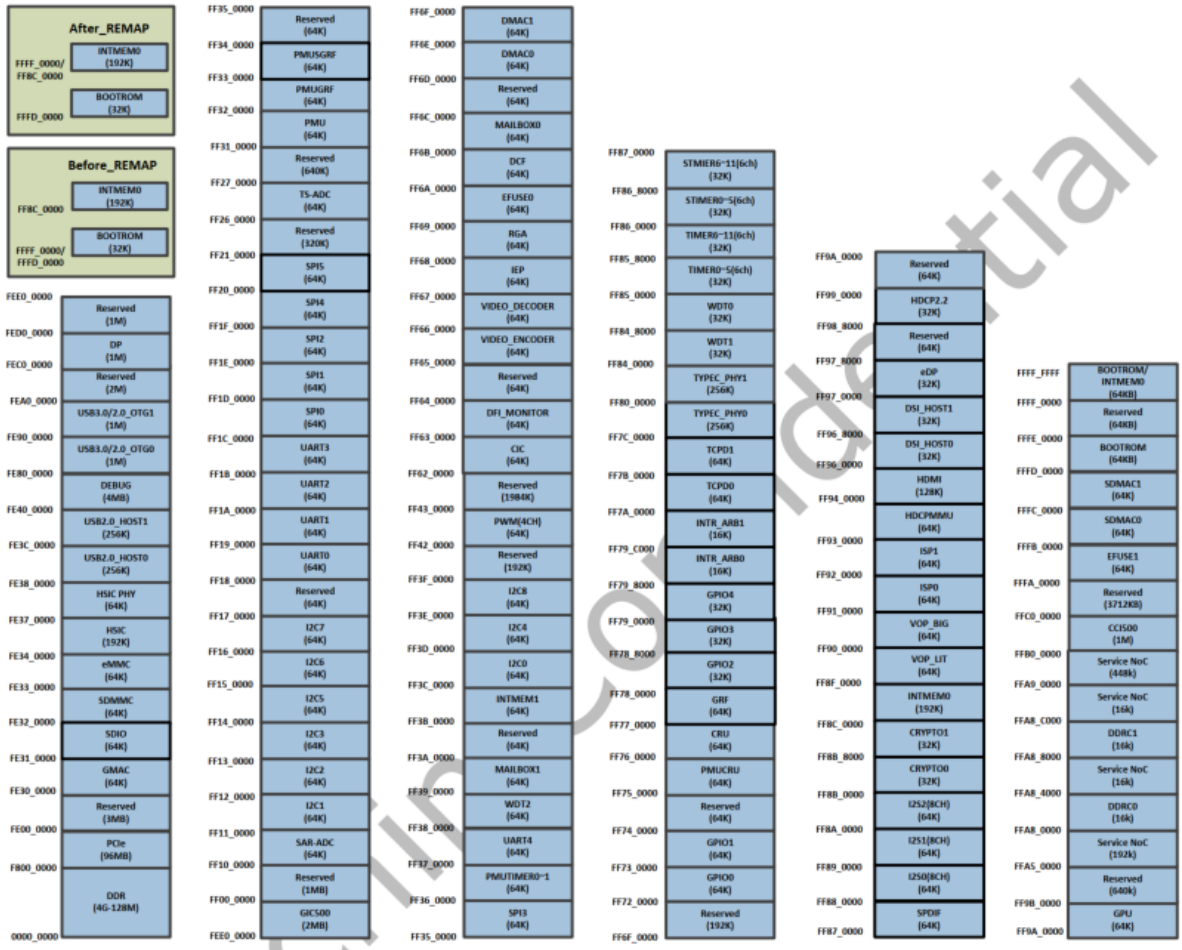


Fig. 2-1 RK3399 Address Mapping

确定GRF的基地址后，加上偏移获得gpio4b0的复用寄存器地址为`0xff77e024`。在串口或者 adb 输入：

```
console:/ # io -4 -l 0x100 0xff77e024
ff77e024: 00000555 0000855f 00000000 00000000
ff77e034: 00000000 00000000 00000000 000002a5
ff77e044: 00005450 0000ff03 00000143 00000000
ff77e054: 00004010 00000001 00000000 0000aa80
ff77e064: 00000455 00002040 0000aaa1 00000000
ff77e074: 00000000 00000000 00000000 00000000
ff77e084: 00000000 00000000 00000000 00000000
ff77e094: 00000000 00000000 00000000 00000000
ff77e0a4: 000000ff 00000000 00000000 00000000
ff77e0b4: 00000000 00000000 00000000 00000000
ff77e0c4: 00000000 00000000 00000000 00000000
ff77e0d4: 00000000 00000000 00000000 00000000
ff77e0e4: 00000000 00000000 00000000 00000000
ff77e0f4: 00000000 00000000 00000000 00000000
ff77e104: 00000000 00000000 00000000 0000b01b
ff77e114: 00000001 00003000 00000000 00000018
```

第一行的结果为：

```
ff77e024: 00000555 0000855f 00000000 00000000
```

四个值分别对应地址：

```
0xff77e024、0xff77e028、0xff77e02c、0xff77e030
```

对应关系(16进制)：

```
0xff77e024：00000555
```

0x555转换为二进制，补齐32位：

```
0000 0000 0000 0000 0000 0101 0101 0101
```

转换后的结果从右至左为低位到高位，即最右边的bit为第0 bit，最左边的 bit为第31 bit。那么对应到之前查询到的寄存器`GRF_GPIO4B_IOMUX`便可以一一对应来查看结果了。

1:0	RW	0x0	gpio4b0_sel GPIO4B[0] iomux select 2'b00: gpio 2'b01: sdmmc_data0 2'b10: uart2dbga_sin 2'b11: reserved

通过查看上述串口输出结果转换二进制后结果，[1:0] bit为01，对应该寄存器的`sdmmc_data0`这个功能。由此，确认了gpio4b0这个IO引脚当前的复用情况。如果这不是用户需要设置的状态，则去代码中查找哪里被复用即可。

==如何查找代码中的复用？==举个例子，例如gpio4b0引脚复用为GPIO功能，作为一个GPIO去拉高拉低，但上述查询结果显示为`sdmmc_data0`功能。那么这时候我们首先要去dts中查找sdmmc节点，先确认节点里是否有pinctrl属性重复引用了该gpio。

IO工具写寄存器操作

如果想要通过IO命令临时写一个寄存器做实验，可以通过`io -w`去写。

举例，已经通过命令`io -4 -r 0xff77e024`读出了寄存器的值，此时想对`0xff77e024`这个寄存器的第0个bit写入1，那么可以如下操作：

注：为什么寄存器地址后面的十六进制值的第16 bit要写1？因为该寄存器的16bit至31bit是写有效位，默认为0，即不可写。因为要往第0 bit写1，所以0bit对应的写有效位16bit也要对应置1才可写入，这个是根据寄存器描述而定的：

GRF_GPIO4B_IOMUX
Address: Operational Base + offset (0x0e024)
GPIO4B iomux control

Bit	Attr	Reset Value	Description
31:16	RW	0x0000	write_enable bit0~15 write enable When bit 16=1, bit 0 can be written by software . When bit 16=0, bit 0 cannot be written by software; When bit 17=1, bit 1 can be written by software . When bit 17=0, bit 1 cannot be written by software; ..... When bit 31=1, bit 15 can be written by software . When bit 31=0, bit 15 cannot be written by software;

Rockchip

需要注意的是，通过IO工具写的寄存器值reboot后不会保留。

IO工具查看GPIO寄存器

想确认当前GPIO到底是输入还是输出、输出高还是输出低等问题，也可以通过读寄存器分析。以gpio4b0举例，通过搜索“ADDRESS MAPPING”在RK3399 TRM上找到gpio4的基地址：



通过io命令查询到 gpio4 的状态，如下：

```
console:/ # io -4 -l 0x100 0xFF790000
ff790000: 42000000 6a400000 00000000 00000000
ff790010: 00000000 00000000 00000000 00000000
ff790020: 00000000 00000000 00000000 00000000
ff790030: ffffffff ffffffff 00000000 00000000
ff790040: 00000000 9434d0f9 00000000 00000000
ff790050: 438b2f06 00000000 00000000 00000000
ff790060: 00000000 00000000 00000000 3230372a
ff790070: 00000000 00000000 00000000 00000000
ff790080: 42000000 6a400000 00000000 00000000
ff790090: 00000000 00000000 00000000 00000000
ff7900a0: 00000000 00000000 00000000 00000000
ff7900b0: ffffffff ffffffff 00000000 00000000
ff7900c0: 00000000 9434d0f9 00000000 00000000
ff7900d0: 438b2f06 00000000 00000000 00000000
ff7900e0: 00000000 00000000 00000000 3230372a
ff7900f0: 00000000 00000000 00000000 00000000
```

对照RK3399 TRM的GPIO章节寄存器介绍来对应查看，比如要看gpio4b0引脚的状态，根据寄存器描述：

10.4.1 Registers Summary

Name	Offset	Size	Reset Value	Description
GPIO_SWPORTA_DR	0x0000	W	0x00000000	Port A data register
GPIO_SWPORTA_DDR	0x0004	W	0x00000000	Port A data direction register
GPIO_INTEN	0x0030	W	0x00000000	Interrupt enable register
GPIO_INTMASK	0x0034	W	0x00000000	Interrupt mask register
GPIO_INTTYPE_LEVEL	0x0038	W	0x00000000	Interrupt level register
GPIO_INT_POLARITY	0x003c	W	0x00000000	Interrupt polarity register
GPIO_INT_STATUS	0x0040	W	0x00000000	Interrupt status of port A
GPIO_INT_RAWSTATUS	0x0044	W	0x00000000	Raw Interrupt status of port A
GPIO_DEBOUNCE	0x0048	W	0x00000000	Debounce enable register
GPIO_PORTA_EOI	0x004c	W	0x00000000	Port A clear interrupt register
GPIO_EXT_PORTA	0x0050	W	0x00000000	Port A external port register
GPIO_LS_SYNC	0x0060	W	0x00000000	Level_sensitive synchronization enable register

*Notes:Size:**B**- Byte (8 bits) access, **HW**- Half WORD (16 bits) access, **W**-WORD (32 bits) access*

0x4偏移为输入输出状态寄存器，而0x0偏移为数据寄存器。查看输入输出状态寄存器`0xff790004`，结果为 0x6a400000，结果对应位为：

gpio4d	gpio4c	gpio4b	gpio4a
0x6a	0x40	0x00	0x00

而每两位十六进制数字转换为二进制数值后从右到左对应0到7脚，比如0x6a二进制是：`0110 1010`，那么二进制从右到左分别对应 gpio4d0 到 gpio4d7 的状态。根据寄存器描述：

RK3399 TRM

Address: Operational Base + offset (0x0004)
Port A data direction register

Bit	Attr	Reset Value	Description
31:0	RW	0x00000000	gpio_swporta_ddr Values written to this register independently control the direction of the corresponding data bit in Port A. 0: Input (default) 1: Output

所以此时gpio4b0是输入状态。假如此时它是输出状态，可通过查看`0xFF790000`的值确认输出的是高还是低电平。如果是输入状态可以通过查看`0xFF790050`（GPIO_EXT_PORTA）确认输入电平状态。

8.1.2 U-Boot阶段IO工具

U-Boot开机阶段长按`ctrl + c`组合键，强制进入命令行。U-Boot命令行模式提供了许多命令，输入`?`可列出当前支持的所有命令：

```
=> ?
?       - alias for 'help'
android_print_hdr- print android image header
base    - print or set address offset
bdinfo  - print Board Info structure
bidram_dump- Dump bidram layout
boot    - boot default, i.e., run 'bootcmd'
……
iomem   - Show iomem data by device compatible(high priority) or node name
……
md      - memory display
……
```

早期版本请参考如下配置重新编译下载U-Boot：

- 使用IO命令需打开以下宏：

```
diff --git a/include/configs/rk_default_config.h
b/include/configs/rk_default_config.h
index 9852917d4f..305de26347 100755
--- a/include/configs/rk_default_config.h
+++ b/include/configs/rk_default_config.h
@@ -304,7 +304,7 @@
/* rk io command tool */
-#undef CONFIG_RK_IO_TOOL
+#define CONFIG_RK_IO_TOOL
```

U-Boot也可以使用md命令

- 使用md命令需添加以下宏：

```
diff --git a/include/configs/rk33plat.h b/include/configs/rk33plat.h
index 0906c0c091..3f94e9eabe 100755
--- a/include/configs/rk33plat.h
+++ b/include/configs/rk33plat.h
@@ -140,6 +140,7 @@
#define CONFIG_RK_GPIO_EXT_FUNC
#define CONFIG_CHARGE_LED
#define CONFIG_POWER_FUSB302
+ #define CONFIG_CMD_MEMORY
#endif
#if defined(CONFIG_RKCHIP_RK322XH)
```

- 设置bootdelay：

```
diff --git a/common/autoboot.c b/common/autoboot.c
index c27cc2c751..fe28adb2fd 100644
--- a/common/autoboot.c
+++ b/common/autoboot.c
@@ -146,6 +146,7 @@ static int abortboot_normal(int bootdelay)
{
int abort = 0;
unsigned long ts;
+ bootdelay = 3; //此处添加时间可自由定义，单位：秒
#ifdef CONFIG_MENUPROMPT
printf(CONFIG_MENUPROMPT);
```

Rockchip平台默认配置已经打开，建议使用md命令读写寄存器。其中==md命令操作==如下：

```
// 读操作
md - memory display
Usage: md [.b, .w, .l, .q] address [# of objects]
// 写操作
mw - memory write (fill)
Usage: mw [.b, .w, .l, .q] address value [count]
```

读操作参考如下：显示`0x76000000`地址开始的连续 0x10 个数据。

```
=> md.l 0x76000000 0x10
76000000: ffffffe ffffffff ffffffff ffffffff ................
76000010: fffffdf ffffffff feffffff ffffffff ................
76000020: ffffffff ffffffff ffffffff ffffffff ................
76000030: ffffffff ffffffff ffffffff ffffffff ................
```

写操作参考如下：对`0x76000000`地址赋值为 0x1234。

```
=> mw.l 0x76000000 0xffff1234 // 高16位有mask
=> md.l 0x76000000 0x10 // 回读
76000000: ffff1234 ffffffff ffffffff ffffffff ................
76000010: fffffdf ffffffff feffffff ffffffff ................
76000020: ffffffff ffffffff ffffffff ffffffff ................
76000030: ffffffff ffffffff ffffffff ffffffff ................
```

8.2 IOMUX工具

iomux是gcc编译的二进制文件，通过ioctl调用rockchip-pinctrl device，可以获取和设置gpio的pin。默认未启动，需要在defconfig中设置CONFIG_ROCKCHIP_IOMUX=y。

iomux工具源码：

```
kernel/tools/testing/selftests/rkpinctrl/iomux.c
```

编译方式：

```
 gcc  kernel/tools/testing/selftests/rkpinctrl/iomux.c -o iomux
```

使用方法：

举例：设置 GPIO0_B7 func1

```
[root@RK3588:/]# iomux 0 15 1
```

举例：获取 GPIO0_B7 iomux

```
[root@RK3588:/]# iomux 0 15
mux get (GPIO0-15) = 1
```

8.3 Pinconfig工具

PinConfig工具为软硬件人员提供统一的IO配置和IO校验功能。目前支持芯片`RK3588|RK3566|RK3568|RV1126|RV1109`。

详细使用手册请看工具中的《PinConfig_UserManual.pdf》文档。

8.4 PinDebug工具

PinDebug工具是一款针对pin的调试工具,可以查询pin的iomux和配置pin的属性，目前支持芯片`RK3588|RK3566|RK3568|RV1126|RV1109|RV1106|RV1103|RK3528` 。

详细使用手册请看工具中的《PinDebug_UserManual.pdf》文档。

9. GPIO 排查步骤

本章介绍GPIO最基本的排查步骤，如遇到GPIO输入输出不受控时，请按照如下步骤排查即可。

9.1 电源确认

测量下该GPIO电源域管脚供电电压是否正常，是否与硬件图纸的设计电压一致？dts中的io-domain配置是否正确？最直接的方式是使用io命令确认下io-domain寄存器，关于电源域配置请参考"IO-Domain章节"。

9.2 IO复用确认

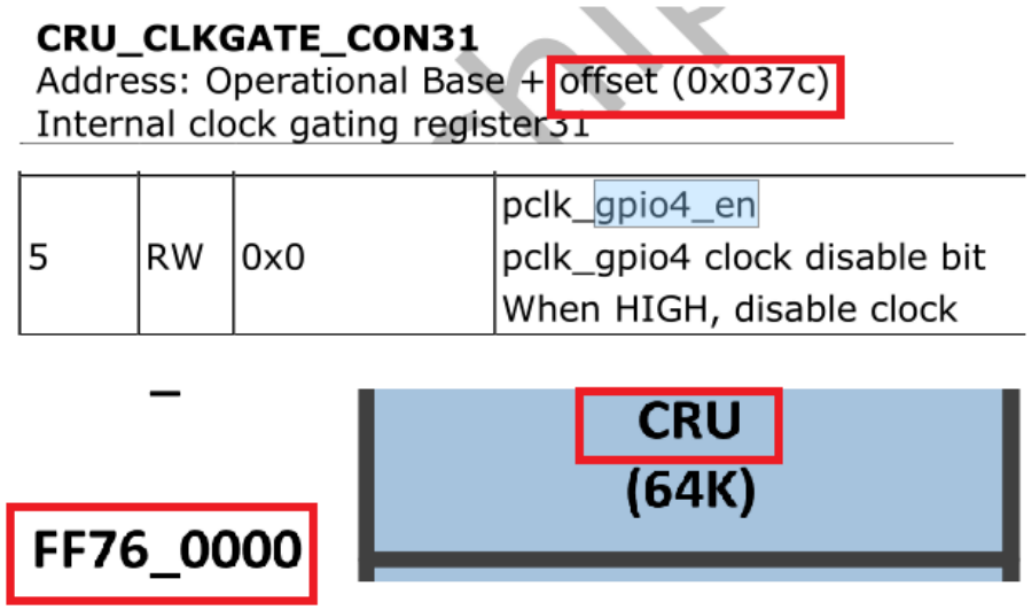
大多数情况都是该GPIO口复用到别的功能导致GPIO无法控制，可以使用GPIO的debug工具排查，一般直接使用io命令确认GPIO复用寄存器，工具的使用请参考"GPIO的debug工具章节"。也可以通过dts的tmp文件，先进行简单的排查。

9.3 GPIO CLK状态确认

当IO工具查看GPIO寄存器时，例如查询gpio4的状态，结果如下：

```
ff790000: 42400000 42400000 42400000 42400000
ff790010: 42400000 42400000 42400000 42400000
ff790020: 42400000 42400000 42400000 42400000
ff790030: 42400000 42400000 42400000 42400000
ff790040: 42400000 42400000 42400000 42400000
ff790050: 42400000 42400000 42400000 42400000
ff790060: 42400000 42400000 42400000 42400000
ff790070: 42400000 42400000 42400000 42400000
ff790080: 42400000 42400000 42400000 42400000
ff790090: 42400000 42400000 42400000 42400000
ff7900a0: 42400000 42400000 42400000 42400000
ff7900b0: 42400000 42400000 42400000 42400000
ff7900c0: 42400000 42400000 42400000 42400000
ff7900d0: 42400000 42400000 42400000 42400000
ff7900e0: 42400000 42400000 42400000 42400000
ff7900f0: 42400000 42400000 42400000 42400000
```

从结果中可以看出，所有的值都奇怪的一致。如果用io命令读出来的值部分或者全部都一致的显示为某个奇怪的值，这是因为 gpio的clk被关闭，无法读写。此时则需把RK3399 gpio4的clk打开：（在RK3399 TRM上逐一搜索gpio4_en）



首先通过io读命令确认当前的gpio4 clk是否为关闭（寄存器bit设置高电平为disable）：

```
io -4 -l  0x100  0xFF76037C
ff76037c: 000001b8 00000010 00000000 0000002a
ff76038c: 00000000 00000000 00000000 00000000
ff76039c: 00000000 00000000 00000000 00000000
ff7603ac: 00000000 00000000 00000000 00000000
ff7603bc: 00000000 00000000 00000000 00000000
ff7603cc: 00000000 00000000 00000000 00000000
ff7603dc: 00000000 00000000 00000000 00000000
ff7603ec: 00000000 00000000 00000000 00000000
ff7603fc: 00000000 00000000 000 00000 00000000
ff76040c: 00000000 00000000 00000000 00000000
ff76041c: 00000000 00000000 00004040 00000000
ff76042c: 00000014 00000000 00000000 00000000
ff76043c: 00000000 00000000 00000000 00000000
ff76044c: 00000000 00000000 00000000 00000000
ff76045c: 00000000 00000000 00000000 00000000
```

```
ff76046c: 00000000 00000000 00000000 00000000
```

结果显示为`0x000001b8`，结果转成二进制也就是：`0001 1011 1000`，那么gpio4 的 clk 对应的第5个 bit，也就是转换成二进制后的数值从右往左数的第5bit 为1（1即disable），那么要把它打开，io命令才能正常读写。用io写命令来打开：

```
io -4 -w  0xFF76037C  0x00200000
io -4 -l 0x100 0xFF790000
ff790000: 42000000 6a400000 00000000 00000000
ff790010: 00000000 00000000 00000000 00000000
ff790020: 00000000 00000000 00000000 00000000
ff790030: ffffffff ffffffff 00000000 00000000
ff790040: 00000000 9434d0f9 00000000 00000000
ff790050: 438b2f06 00000000 00000000 00000000
ff790060: 00000000 00000000 00000000 3230372a
ff790070: 00000000 00000000 00000000 00000000
ff790080: 42000000 6a400000 00000000 00000000
ff790090: 00000000 00000000 00000000 00000000
ff7900a0: 00000000 00000000 00000000 00000000
ff7900b0: ffffffff ffffffff 00000000 00000000
ff7900c0: 00000000 9434d0f9 00000000 00000000
ff7900d0: 438b2f06 00000000 00000000 00000000
ff7900e0: 00000000 00000000 00000000 3230372a
ff7900f0: 00000000 00000000 00000000 00000000
```

打开clk除了上述写寄存器操作，还可以通过sys接口操作，如：

```
console:sys/kernel/debug/clk # cat clk_summary | grep gpio3
pclk_gpio3         0        1        0   100000000          0     0  50000
console:sys/kernel/debug/clk # ls pclk_gpio3
pclk_gpio3/clk_accuracy         pclk_gpio3/clk_parent
pclk_gpio3/clk_available_parent pclk_gpio3/clk_phase
pclk_gpio3/clk_duty_cycle       pclk_gpio3/clk_prepare_count
pclk_gpio3/clk_enable_count     pclk_gpio3/clk_protect_count
pclk_gpio3/clk_flags            pclk_gpio3/clk_rate
pclk_gpio3/clk_notifier_count

console:sys/kernel/debug/clk # echo 1 > pclk_gpio3/clk_enable_count
console:sys/kernel/debug/clk # cat clk_summary | grep gpio3
    pclk_gpio3          1        2        0   100000000          0     0  50000
```

注意：gpio clk是否打开取决于对应gpio是否处于活跃状态(如中断或数据传输中)，而与是否配置或调用无直接关联。

9.4 其他驱动调用GPIO导致冲突确认

比如一个IO引脚，想作为GPIO使用，按照上面方法确认了复用没错，并且电源域确认也没问题，但是示波器测量波形感觉没有按照自己驱动代码来拉高拉低，有些“不受控”。这时候很可能就是有其它驱动在操作这个GPIO，可以如下在`gpio_direction_output`和`gpio_direction_input`接口函数中添加一些调试信息，将所有操作GPIO的信息打印出来。

```
drivers/gpio/gpiolib.c
int gpiod_direction_input(struct gpio_desc *desc)
{
    + pr_info("%s:  %d\n", __func__, desc_to_gpio(desc));
    ......
}

int gpiod_direction_output(struct gpio_desc *desc, int value)
{
    + pr_info("%s:  %d\n", __func__, desc_to_gpio(desc));
    ......
}
```

以RK3588 gpio4b3 来举例，排查“不受控”原因。首先，通过如下命令确认gpio4a0的引脚首编号：

```
root@rk3588-buildroot:/# cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 0-31, parent: platform/fd8a0000.gpio, gpio0:
 gpio-26  (                    |GTP_RST_PORT        ) out hi
 gpio-27  (                    |GTP_INT_IRQ         ) in  hi

gpiochip1: GPIOs 32-63, parent: platform/fec20000.gpio, gpio1:
 gpio-41  (                    |vcc-mipicsi0-regulat) out lo
 gpio-42  (                    |vcc-mipicsi1-regulat) out lo

gpiochip2: GPIOs 64-95, parent: platform/fec30000.gpio, gpio2:
 gpio-76  (                    |reset               ) out hi ACTIVE LOW
 gpio-77  (                    |hdmirx-det          ) in  hi ACTIVE LOW
 gpio-84  (                    |vcc-mipidcphy0-regul) out hi

gpiochip3: GPIOs 96-127, parent: platform/fec40000.gpio, gpio3:
 gpio-96  (                    |bt_default_wake_host) in  lo
 gpio-113 (                    |ircut-open          ) out lo
 gpio-115 (                    |vcc3v3-pcie30       ) out lo

#gpio4a0的引脚首编号为128
gpiochip4: GPIOs 128-159, parent: platform/fec50000.gpio, gpio4:
 gpio-130 (                    |reset               ) out hi
 gpio-133 (                    |reset               ) out hi
 gpio-134 (                    |sbu1-dc             ) out lo
 gpio-135 (                    |sbu2-dc             ) out hi
 gpio-136 (                    |vcc5v0-host         ) out hi
 gpio-137 (                    |enable              ) out lo
 gpio-138 (                    |enable              ) out lo
 gpio-152 (                    |vbus5v0-typec       ) out lo
```

计算gpio4b3的引脚编号：

```
gpio4b3 = 128 + (B-A) * 8 + 3 = 128 + 1 * 8 + 3 = 139
```

排查是否有139的信息：

```
root@rk3588-buildroot:/# dmesg | grep "gpiod_direction_output"
[    1.460212] gpiod_direction_output:  152
[    1.460291] gpiod_direction_output:  115
[    1.460391] gpiod_direction_output:  136
......
[    1.735094] gpiod_direction_output:  133
[    1.806359] gpiod_direction_output:  139  #有驱动控制了gpio4b3
[    2.136271] gpiod_direction_output:  113
```

9.5 GPIO 寄存器确认

如果IO电源域和IO复位确定都没有问题，可使用io命令直接读写GPIO寄存器，最终确认是否是软件配置问题。

需排查如下3个GPIO寄存器：

Port Data Register：GPIO输出寄存器(Low/High)

Port Data Direction Register：GPIO方向寄存器(Input/Output)

External Port Data Register：GPIO外部端口电平值(Low/High)

如配置一个GPIO输出为高电平，经排查GPIO输出寄存器配置为1，GPIO方向寄存器配置为1，但是GPIO外部端口电平值为0，此时基本判定是硬件原因，外围的电路把该GPIO拉低了。

10. FAQ

10.1 系统上电输出电平无法控制

如果需要上电就输出高电平或者低电平可控，这个软件是不可控制的。需要选择默认硬件状态上拉或者下拉的GPIO引脚。以RK3399 gpio2c2引脚举例(无外围电路)，对默认上电状态有要求的gpio，可通过datasheet或原理图命名后缀字母确认引脚默认状态：

如想要选择默认上电为高电平的GPIO，请选择下标为-u的GPIO管脚，如要选择默认上电为低电平的GPIO，请选择下标为-d的GPIO管脚。

注意：有些管脚默认不是复用在GPIO状态，可以通过查询相应TRM文档的复用寄存器的默认状态。比如JTAG引脚，寄存器默认配置为JTAG模式。

10.2 GPIO中断误触发

GPIO中断如果有误触发，请打开施密特触发模式，即gpio配置为pcfg_pull_none_smt模式。

10.3 RV1103/RV1106的gpio3b0~gpio3c3无法控制

RV1103/RV1106的gpio3b0~gpio3c3特殊，如需配置为GPIO模式，还需特殊配置相关寄存器，并且只能用作输入（这组IO的GPIO和MIPI模式只能同时生效一种）。按照如下步骤：

```
io -4 0xFF3B4804 0x40000000 #使能gpio3b的clk
io -4 0xFF3B4808 0x00100000 #使能gpio3c的clk
io -4 0xFF3E8000 0x7D #Clk/data lane enable
io -4 0xFF3E8080 0x5F #MIPI/LVDS enable

# 如果引脚是gpio3b6~gpio3b7和gpio3c0~gpio3c，则配置0xFF3E844C寄存器
io -4 0xFF3E844C 0x00010001 #切换csi_dphy_path0为TLL_MODE
# 如果引脚是gpio3b0~gpio3b5，则配置0xFF3E884C寄存器
io -4 0xFF3E884C 0x00010001 #切换csi_dphy_path1为TLL_MODE

io -4 0xFF55804c 0x00070000 #设置GPIO3_B4复用寄存器
```

10.4 RV1108的gpio1a0~gpio1b1无法控制

RV1108的gpio3b0~gpio3c3特殊，如需配置为GPIO模式，需配置DPHY_LVDS_TTL为TTL mode，如下设置：

```
io -4 0x2022838C 0x00000004 #enable TTL mode，disable LVDS mode，disable mipi mode
io -4 0x202283AC 0x000000fd #disable Data Lane，disable Clock Lane
```

10.5 RK3588的gpio4d2、gpio4d3无法控制

以rk3588为例（其他如rk3576平台有此问题也可参考），如果需要配置为GPIO模式，需要查看手册force_jtag，如图：

SYS_GRF_SOC_CON6

Address: Operational Base + offset (0x0318)

Bit	Attr	Reset Value	Description
31:16	WO	0x0000	write_enable Write enable for lower 16bits, each bit is individual 1'b0: Write access disable 1'b1: Write access enable
15	RW	0x0	pciephy_dtb_sel PCIEPHY DTB select 1'b0: Select PCIE0 PHY DTB output to IO 1'b1: Select PCIE1 PHY DTB output to IO
14	RW	0x1	force_jtag Force SDMMC IO to JTAG function enable When force_jtag is high , gpio4_d2_sel is 0x1 , gpio4_d3_sel is 0x1, and SDMMC_DETECTN is inactive, switch the SDMMC IO to CPU JTAG function.

需要将默认的force_jtag位关闭:

```
io -4 0xFD58C318 0x40000000 #disable force_jtag
io -4 0xFD5F8098 0x0F000000 #iomux set to gpio
```

10.6 Android SDK没有找到/sys/class/gpio的系统层接口

在 device/rockchip/rk3588/BoardConfig.mk 文件中修改 BOARD_BUILD_GKI := false
内核配置开启宏 CONFIG_DEBUG_GPIO和CONFIG_GPIO_SYSFS

10.7 GPIO读取电平异常

1. IOMUX是否配置为GPIO
2. GPIO的DDR寄存器是否为输入模式
3. GPIO的clk是否开启
4. GPIO是否内部/外部上拉
5. GPIO读GPIO_EXT_PORT寄存器查看电平状态（这个寄存器体现的是pin的实际电平，因此需确保IOMUX、DDR配置正确，且上下拉是非高阻态）

10.8 GPIO中断绑定指定核心

参照下面补丁，可修改中断亲和性到CPU2

```
--- a/drivers/gpio/gpio-rockchip.c
+++ b/drivers/gpio/gpio-rockchip.c
@@ -579,6 +579,25 @@ static int rockchip_interrupts_register(struct
rockchip_pin_bank *bank)
        irq_set_chained_handler_and_data(bank->irq,
                                         rockchip_irq_demux, bank);

+    // +++ 新增：设置中断亲和性到CPU2 +++
+    {
+        struct cpumask cpumask;
+        int cpu = 2; // 指定CPU2
+
+        // 检查CPU2是否有效
+        if (cpu < num_possible_cpus()) {
+            cpumask_clear(&cpumask);
+            cpumask_set_cpu(cpu, &cpumask);
+            if (irq_set_affinity_hint(bank->irq, &cpumask)) {
+                dev_warn(bank->dev, "Failed to set IRQ affinity for CPU%d\n",
cpu);
+            } else {
+                dev_warn(bank->dev, "IRQ%d affinity set to CPU%d\n", bank->irq,
cpu);
+            }
+        } else {
+            dev_warn(bank->dev, "Invalid CPU%d for IRQ affinity\n", cpu);
+        }
+    }
+
        return 0;
 }
```

10.9 GPIO中断响应延迟

1. 修改中断亲和性
2. CONFIG_HZ=1000 //修改心跳间隔

```
• 高 HZ 值 (如 1000):
每 1ms 触发一次时钟中断，调度器有最多 1ms 延迟响应新任务。
• 低 HZ 值 (如 100):
每 10ms 触发一次中断，任务切换延迟可能高达 10ms。
```

3. 提高CPU频率
4. 打上实时性补丁，可在docs\Patches\Real-Time-Performance根据里面的文档打上补丁